

NumPy and Matplotlib

EEC 3150

Trevecca Nazarene University

1 NumPy

- Intro to NumPy
- Creating ndarray's
- Math operations on ndarray's
- Indexing ndarray's

NumPy: Numerical Python

NumPy:

NumPy is one of the most commonly used third-party libraries in Python

<https://numpy.org/doc/stable/>

Key features:

- Extensive library of tools for mathematical computations
 - Trigonometric, exponential, and special functions
 - Statistics and random number generation
 - Linear algebra and Fourier analysis (and more)
- Bread and butter: `ndarray` objects
 - n -dimensional arrays
 - Lists but infinitely better (how you wish lists worked)

Importing numpy:

```
import numpy as np
```

Creating ndarray's

Functions for creating arrays:

- `np.array(x)` will convert the list `x` into an array
The function accepts an additional keyword argument `dtype` that allows for the specification of data type the array should have (ex: `np.int32`, `np.float64`)
- `np.zeros(n)` will create an array of length `n` (int) of all zeros
You can also pass a tuple instead of an integer to create a multi-dimensional array (ex: `np.zeros((m,n))` creates an $m \times n$ array)
- `np.ones(n)` (like above, but all ones)
- `np.full(n,value)` (like above, but fills array with `value`)
- `eye(n)` creates an $n \times n$ identity matrix
- `np.arange(a,b,step)` same as Python `range` but allows floats
- `np.linspace(a,b,n)` creates an array of `n` evenly-spaced points from `a` to `b`
- `np.random.rand(n)` creates a length `n` array of random numbers b/t 0 and 1
This uses the `np.random` library (more on that later)

Some additional points:

- The different dimensions of a numpy array are called *axes* (`axis` is often a keyword argument to many functions)
- Use the `shape` attribute to get the length of an array along each axis (ex: `x.shape`)

Creating ndarray's

2 ways of creating a 3×3 array of all 5's:

- `np.full((3,3),5)`
- `5*np.ones((3,3))`

Row vs. column vs. 1-D:

These are all different objects:

- `np.ones(5)`
This is a 1-D array
- `np.ones((5,1))`
This is a 2-D array (column)
- `np.ones((1,5))`
This is a 2-D array (row)

Math operations ndarray's

Math operations ndarray's:

Numpy arrays can handle mathematical operations in an intuitive, element-wise way:

- We can perform virtually any mathematical operation on any combination of arrays and scalars (as long as their lengths in each dimension match)
- The exception to this is Boolean operations, but Numpy has functions for these anyway
- We can also apply virtually any of Numpy's mathematical functions on arrays in an element-wise fashion
- Numpy can even add arrays of different sizes in certain contexts

Math operations ndarray's

Plot some random numbers:

- Create an array of random numbers $b/t - a$ and a
- Create and plot a linearly-spaced array of numbers with the above random numbers added as noise

Plot sin and cos:

Plot $\sin(x)$ and $\cos(x)$ from 0 to 2π

Indexing ndarray's

Indexing ndarray's

Standard list indexing:

- Index an element: `x[1]`
- Get a slice: `x[4:12]`
- Index from the end: `x[-3]`

Numpy also supports *fancy indexing*:

- Indexing with `np.newaxis` will add another axis to the array
- Index a 2-D array: `x[2,5]`
- Index an entire row, column respectively: `x[2,:]`, `x[:,5]`
- Index a list of specific elements: `x[[1,4,8]]`
- Index all elements with values greater than 0: `x[x>0]`
 - How it works: `x>0` creates a boolean mask used to index the `x` array
 - This works for multi-dimensional arrays too
 - Limitation: you can't do multiple conditionals in the same fancy index
- Get the indices (not values) where an array is greater than 0: `np.where(x>0)`

In addition to accessing array elements with the above techniques, you can also SET elements with the above techniques

Conditional masking vs. `np.where()`

Let `x` be a numpy array:

- `x[x>0]` returns the VALUES where `x` is greater than 0
- `np.where(x>0)` returns the INDICES where `x` is greater than 0

Indexing ndarray's

Set negative elements to 0:

- Generate some random data
- Set all negative values to 0

Index with multiple conditions:

- Generate some random data
- Using several boolean masks, index the values such that $a < x < b$
- There are several ways to do this
 - Use numpy's `np.logical_and(x,y)` function
 - Repeatedly index and overwrite the same array with different boolean masks

Plot positive $\sin(x)$:

- Create `x,y` arrays for plotting $\sin(x)$ from $0, 4\pi$
- Only plot values where $\sin(x) \geq 0$
- Plot with dots